Architecture Client-Serveur et Fondements du Protocole HTTP

Thème principal : Fonctionnement du Web, Protocole HTTP, Méthodes GET/POST et Codes de Statut

Synthèse exécutive : Ce cours pose les bases fondamentales du développement web en décortiquant le modèle de communication client-serveur et le protocole HTTP. Il explique comment un navigateur interagit de manière asymétrique et sans état (stateless) avec un serveur web, la manière dont la persistance est gérée via des cookies de session, la structure précise des requêtes et réponses HTTP, la distinction critique entre les méthodes GET et POST, ainsi que la signification des codes de statut essentiels pour le débogage.

1. L'Architecture Client-Serveur et la Nature Stateless de HTTP

L'architecture client-serveur est la structure fondamentale du Web. Le client (un navigateur tel que Firefox, Chrome ou Safari) est le composant actif : il formule et envoie une requête HTTP à travers le réseau IP. Le serveur web (un démon applicatif distant tel qu'Apache ou Nginx) est l'élément passif : il écoute en continu sur un port dédié (port 80 pour HTTP, port 443 pour HTTPS). Lorsqu'il reçoit une requête, il la traite — ce qui peut nécessiter l'accès à des fichiers locaux ou l'exécution de scripts interrogeant une base de données — puis retourne une réponse au client. HTTP est un protocole 'stateless' (sans état). Cela signifie que le serveur ne conserve aucun souvenir des interactions passées. Sans mécanisme de contournement, chaque clic réinitialiserait l'application (perte du panier d'achat, déconnexion systématique). Pour résoudre ce problème, on utilise des cookies de session : lors de la première connexion, le serveur génère un identifiant unique (Session ID) transmis au client. Le navigateur stocke ce cookie et le réémet automatiquement dans l'en-tête de chaque requête ultérieure, permettant au serveur de reconstituer le contexte de l'utilisateur.

- Asymétrie Client-Serveur : Le client (navigateur) prend toujours l'initiative d'émettre une requête ; le serveur attend passivement sur un port (80 pour HTTP, 443 pour HTTPS) et renvoie une réponse.
- Protocole Stateless (Sans état) : Propriété de HTTP où chaque échange est totalement indépendant, le serveur ne conservant aucune mémoire des requêtes précédentes.
- Gestion de Session (Cookies) : Mécanisme d'en-tête permettant d'injecter un identifiant de session (Session ID) pour réidentifier l'utilisateur à chaque requête et simuler un état persistant.

Cas concret enseignant : *Le professeur compare HTTP à un guichetier amnésique : à chaque fois que vous vous présentez devant lui, il vous traite comme un inconnu. Pour qu'il se souvienne de vous, vous devez lui présenter votre badge (le cookie de session) à chaque phrase. De plus, lors d'une visite sur le site de l'IUT, le navigateur n'envoie pas une seule requête, mais des dizaines de requêtes HTTP simultanées pour récupérer le fichier HTML de base, puis chaque feuille CSS, script JS et image.*

2. Structure Interne des Messages HTTP et Comparatif GET vs POST

Les messages HTTP sont des structures textuelles lisibles découpées en 4 parties distinctes : 1. La ligne de requête (Méthode + URI + Version HTTP) ; 2. Les en-têtes (Host obligatoire, User-Agent, etc.) ; 3. Une ligne vide servant de séparateur strict (CRLF) ; 4. Le corps (Body), contenant les données soumises. Le choix de la méthode est fondamental : - GET est conçue uniquement pour lire des données sans modifier l'état du serveur. Les variables sont transmises dans l'URL (`?clé=valeur`). GET ne doit jamais être employée pour des données sensibles. - POST est conçue pour envoyer des données modifiant l'état du serveur (création de compte, validation). Les données transitent dans le corps du message, ce qui les masque dans la barre d'adresse et permet le transfert de gros volumes (fichiers, images).

- Format Textuel Normalisé : Un message HTTP est lisible en clair et comprend une ligne de commande/statut, des en-têtes (headers), une ligne vide (CRLF) et un corps (body).
- Méthode GET : Utilisée pour consulter des ressources. Elle est sûre, idempotente, et transmet ses paramètres directement dans l'URL (Query String). Sa taille est limitée (~2048 caractères).

• Méthode POST : Utilisée pour soumettre des données modifiant l'état du serveur. Les paramètres sont encapsulés dans le corps (body), sans restriction de taille et invisibles dans l'URL.

Cas concret enseignant : *Le professeur alerte sur l'erreur classique d'envoyer un formulaire de connexion en GET : le mot de passe s'affiche en clair dans la barre d'adresse de l'écran, s'enregistre dans l'historique du navigateur et apparaît en clair dans les logs du serveur. C'est une vulnérabilité majeure au 'shoulder surfing' (regard par-dessus l'épaule).*

3. Décodage et Classification des Codes de Statut HTTP

Chaque réponse HTTP débute par une ligne de statut contenant un code numérique à 3 chiffres classé par familles : - 1xx (Information) : Traitement en cours. - 2xx (Succès) : La requête a été acceptée et traitée (ex: 200 OK, 201 Created). - 3xx (Redirection) : La ressource a changé d'emplacement (ex: 301 Moved Permanently, 304 Not Modified). - 4xx (Erreur Client) : Erreur dans la formulation de la requête ou droits insuffisants (ex: 401 Unauthorized, 403 Forbidden, 404 Not Found). - 5xx (Erreur Serveur) : La requête était valide, mais le serveur a échoué lors du traitement (ex: 500 Internal Server Error, 503 Service Unavailable).

• Ligne de Statut : Première ligne d'une réponse HTTP contenant la version du protocole, un code numérique à trois chiffres et un message textuel explicatif.

• Familles de Codes : 1xx (Info), 2xx (Succès), 3xx (Redirection), 4xx (Erreur Client), 5xx (Erreur Serveur).

• Responsabilité du Débogage : Permet de déterminer instantanément si l'erreur vient de la requête formulée par le client (4xx) ou d'un dysfonctionnement de l'infrastructure/code backend (5xx).

Cas concret enseignant : *Le professeur raconte qu'à ses débuts, une erreur de syntaxe PHP (point-virgule manquant) empêchait la connexion à la base de données. Le serveur renvoyait un code 500. Les utilisateurs pensaient que le site avait été piraté alors qu'il s'agissait simplement d'une faute de frappe dans le code backend.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Monsieur, quelle est la différence concrète entre GET et POST pour envoyer un formulaire de connexion ?
Réponse du professeur : Avec GET, les paramètres de votre formulaire (identifiant ET mot de passe) sont directement accolés à l'URL. Le mot de passe apparaît en clair dans la barre d'adresse, reste stocké dans l'historique du navigateur et est écrit dans les fichiers de logs du serveur. Avec POST, les données sont encapsulées dans le corps (body) de la requête HTTP : elles n'apparaissent pas dans l'URL et ne sont pas conservées dans l'historique, ce qui est indispensable pour la sécurité.

■ Exercice & Démonstration d'Application

Énoncé : Analyser la requête HTTP brute suivante. Identifier la méthode utilisée, la ressource demandée, l'hôte cible, et expliquer le comportement du serveur avec le code de retour exact si la ressource sollicitée n'existe pas sur le disque : GET /images/logo.png HTTP/1.1 Host: iutbayonne.univ-pau.fr User-Agent: Mozilla/5.0

Résolution pas à pas :
1. Ligne de requête ('GET /images/logo.png HTTP/1.1') : - Méthode : GET (demande de lecture/récupération d'une ressource sans modification de l'état du serveur). - Ressource (URI) : '/images/logo.png' (chemin absolu de la ressource recherchée). - Version du protocole : HTTP/1.1. 2. En-tête Host ('Host: iutbayonne.univ-pau.fr') : - Nom de domaine du serveur destinataire. Obligatoire en HTTP/1.1 pour permettre l'hébergement virtuel (Virtual Hosting) de plusieurs sites sur la même IP. 3. En-tête User-Agent ('User-Agent: Mozilla/5.0') : - Identifie le logiciel/navigateur client effectuant la requête. 4. Comportement si le fichier n'existe pas : - Le chemin demandé pointe vers une ressource inexistante. La responsabilité de l'erreur incombe au client ayant demandé une mauvaise URI. - Le serveur renverra un code de statut de la famille 4xx, spécifiquement '404 Not Found'.

Résultat : Méthode : GET | Ressource : /images/logo.png | Hôte : iutbayonne.univ-pau.fr | Code si fichier absent : 404 Not Found (Erreur de responsabilité Client).

■ Auto-Évaluation Flash d'Examen

Q : Qu'est-ce qu'un protocole 'stateless' comme HTTP et quelle est sa conséquence directe sur la gestion des sessions ?

R : Un protocole 'stateless' ne conserve aucun souvenir des requêtes précédentes. Chaque échange est isolé. Sa conséquence directe est qu'il faut utiliser des mécanismes externes (comme des cookies contenant un Session ID) pour identifier l'utilisateur à chaque nouvelle requête et simuler un état persistant.

Q : Quelle est la différence fondamentale de responsabilité entre une erreur HTTP de la série 4xx et une erreur de la série 5xx ?

R : La série 4xx indique une erreur de la responsabilité du client (ex: mauvaise URL, ressource inexistante, absence d'authentification). La série 5xx indique une erreur de la responsabilité du serveur (ex: crash d'un script backend, serveur saturé, erreur de syntaxe PHP).

Q : Pourquoi la méthode HTTP GET ne doit-elle jamais être employée pour transmettre un mot de passe ?

R : Parce que GET transmet les données dans l'URL. Le mot de passe devient visible à l'écran (risque de shoulder surfing), reste enregistré dans l'historique de navigation du client et est conservé en clair dans les journaux d'accès (logs) du serveur web.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765